End-to-End Continuous Testing Strategy & Capstone Preparation

Day 8 · Module 8 · 2 Hours · Final Session

SDET / DevOps Engineer

Intermediate

30% Strategy / 70% Capstone

TODAY

- Integrate Days 1–7
- Capstone Project Brief
- Strategy Document
- Peer Review

SESSION GOALS

What Today's Session Achieves

Synthesise all seven modules into a coherent strategy

Walk through how the course's seven topics (frameworks, version control, CI pipelines, API testing, containers, reporting, optimisation) connect into a single, deliverable continuous testing practice.

Produce a Capstone Project — end-to-end pipeline design

Design and document a complete CI pipeline for a sample application: test strategy, tool choices, pipeline stages, environment configuration, quality gates, and reporting approach.

Present, peer-review, and refine the strategy document

Each participant or team presents their pipeline design. Peers provide structured feedback. Trainer facilitates alignment to industry best practices.

COURSE JOURNEY

The 8-Day Journey — What We Built

D1 CT Fundamentals

Shift-left/right · Test pyramid · DORA · CI/CD pipeline stages

D2 Test Frameworks

JUnit 5 · PyTest · TestNG · Maven/Gradle · CI-friendly design

D3 Version Control

Git workflows · PR best practices · Branch protection · CODEOWNERS

D4 CI Pipeline Design

GitHub Actions · Jenkins · Azure DevOps · Parallel/matrix · Flaky tests

D5 API Testing

REST Assured 6 · Postman/Newman · Contract/schema validation · CI integration

D6 Containers & Data

Docker · Testcontainers · Test data strategies · Env config

D7 Reports & Gates

Allure 3 · JaCoCo 0.8.14 · Coverage thresholds · Pipeline optimisation

D8 Strategy & Capstone

End-to-end integration · Pipeline design doc · Peer review

End-to-End Continuous Testing Strategy Framework

Test Portfolio

- Unit (60–70%) — fast, isolated
- Integration (20–30%) — service boundary
- API (service-level contract)
- E2E (≤10%) — critical journeys only
- Avoid over-indexing on E2E — slow & fragile

Pipeline Architecture

- Trigger strategy: push → unit; PR → full suite
- Fail fast: unit tests first (needs:)
- Parallel matrix for integration tests
- Quality gates: JaCoCo + Allure trend
- Artefact upload + reporting action

Toolchain Decisions

- Framework: JUnit 5 / PyTest / TestNG
- API: REST Assured / Newman
- Containers: Testcontainers / Compose
- Reports: Allure 3 + JaCoCo 0.8.14
- CI platform: GitHub Actions / Jenkins / AzDO

Quality Gates

- Coverage: LINE ≥ 80%, BRANCH ≥ 70%
- Pass rate target > 95% per run
- Flake rate < 2% (quarantine policy)
- All CI steps: if: always() for reports
- Block merge on gate failure

Metrics & Feedback

- Pipeline duration < 10 min target
- Test pass rate trend (Allure history)
- Mean time to green — track weekly
- Coverage trend — enforce via JaCoCo check
- Flaky test SLA — assign owner, fix or quarantine

Capstone Project Brief — Design Your CI Pipeline

🎯 Scenario: You are joining a team as SDET / DevOps Engineer. The team has a Java Spring Boot REST API with no automated CI pipeline. Your task: design and document a complete continuous testing pipeline from scratch using the tools and practices from this course.

01 Test Strategy Document

- Test pyramid allocation — % split and rationale for your architecture
- Tool choices — framework, API testing tool, CI platform, reporting tool
- Quality gate definitions — coverage thresholds, pass rate target, flake policy
- Test data strategy — seed scripts, UUID isolation, environment config approach

02 CI Pipeline Design

- Pipeline YAML skeleton (GitHub Actions / Jenkins / Azure DevOps — your choice)
- Job dependency graph: which jobs are parallel, which require needs:
- Trigger strategy: push / PR / schedule / merge — what runs when
- Report publishing: Allure history + JaCoCo gate integrated into pipeline

03 Environment & Data Configuration

- Docker / Testcontainers strategy for integration tests
- application-test.properties with env-var injection pattern
- Seed data approach and secret management (no hardcoded credentials)
- Branch protection rules and CODEOWNERS definition for the test repo

Capstone Evaluation Criteria

Scoring Rubric

20 — **Test Strategy**

Clear pyramid rationale, appropriate tool choices, documented quality gate definitions

25 — **Pipeline Design**

Correct YAML structure, logical job dependencies, appropriate trigger strategy, parallel execution where applicable

20 — **Env & Data Config**

Docker/Testcontainers usage, env-var injection, no hardcoded secrets, test data isolation strategy

20 — **Reports & Gates**

Allure integration, JaCoCo coverage gate configured, reporting step with if: always()

15 — **Presentation & Peer Review**

Clear delivery, ability to justify decisions, constructive peer feedback given

Peer Review Guide

What works well?

Start with genuine strengths: a well-structured pipeline graph, a clear data strategy, good use of fail-fast sequencing. Be specific, not generic.

What would you question?

Probe decisions: 'Why E2E tests on every PR push?' / 'How does the coverage threshold handle legacy code?' / 'What happens when the DB container doesn't start?'

What would you improve?

Offer one concrete suggestion with rationale: 'Adding a nightly schedule for E2E would remove them from the PR gate and reduce PR wait time.'

Presentation checklist

5 min presentation + 5 min Q&A per participant. Walk through: scenario → decisions → pipeline diagram → quality gate → what you would iterate next.

Common CI Testing Anti-Patterns — What to Avoid

✕ Ice-cream cone test distribution

More E2E than unit tests. Slow, brittle, expensive to maintain. Flip the pyramid: invest heavily at unit and service layers; keep E2E thin and stable for critical journeys only.

✕ Hardcoding environment config

Base URLs, DB credentials, or API keys committed to test code or compose files. Use env-var injection and CI secrets exclusively. Violates security and portability principles.

✕ Over-relying on retries for flaky tests

Retries mask flakiness — they do not fix it. A retry culture accumulates technical debt and erodes CI trust. Detect → quarantine → assign owner → fix.

✕ Targeting 100% coverage

100% coverage leads to tests that verify implementation details rather than behaviour. Set risk-based thresholds (70–85% typical; 85–95% critical logic). Exclude generated/config code.

✕ Full E2E suite on every PR push

E2E tests are slow and environment-sensitive. Run fast unit + API tests on PR push. Reserve full regression for scheduled/nightly runs or post-merge to main.

✕ No test reports on failure

Not using if: always() on report steps means the report disappears when you need it most. Every Allure, JUnit XML upload, and JaCoCo publish step must use if: always().

CAPSTONE LAB

Capstone Work Session — Build Your Pipeline Design

70% of today's session

Phase 1
30 min

Test strategy section of design doc

Draft Test Strategy & Tool Selection

1. Define your test pyramid split: what % unit / integration / API / E2E and why
2. Choose your CI platform (GitHub Actions recommended) and justify the choice
3. Define quality gates: coverage %, pass rate target, flake policy
4. Choose your reporting toolchain: Allure + JaCoCo — configure dependency versions

Phase 2
30 min

Pipeline YAML skeleton (ci.yml)

Design CI Pipeline YAML Structure

1. Sketch job graph: unit-test → api-test (parallel) → integration-test → report
2. Write GitHub Actions YAML skeleton: triggers, jobs, needs:, runs-on:
3. Add: checkout@v4, setup-java@v5 (temurin 21), cache@v4, upload-artifact@v4
4. Wire in: mvn test → mvn verify → mvn allure:report → peaceiris gh-pages publish

Phase 3
20 min

Peer feedback form + refined strategy doc

Peer Review & Presentation

1. 5-min walk-through: scenario → decisions → pipeline diagram → quality gate
2. Peer asks: one strength, one question, one suggestion (structured feedback)
3. Trainer facilitates: aligns to patterns seen across Days 1–7
4. Capture: what would you iterate next? Add to your strategy doc as next steps

DELIVERABLES

What You Take Away From This Course

📦 Complete Course Deliverable Portfolio

D1 CI pipeline concept map — stages, triggers, test levels, shift-left/right

D2 CI-ready test automation framework skeleton (Maven / JUnit 5 / TestNG)

D3 Git repository with test code, branch protection rules, CODEOWNERS

D4 CI pipeline configuration file (GitHub Actions / Jenkins / Azure DevOps)

D5 API test suite (REST Assured + JSON Schema) integrated into CI pipeline

D6 Dockerfile + docker-compose.test.yml + environment configuration documentation

D7 Published Allure report + JaCoCo coverage gate evidence

D8 Capstone: End-to-end CI pipeline design document + peer-reviewed strategy

KEY TAKEAWAYS

Course Key Takeaways & Your Next Steps

The Big Ideas

1. Continuous testing is a culture, not a tool — every team member owns quality
2. The test pyramid reduces CI time and flakiness — invest at unit and service layers
3. Pipeline-as-code means test strategy is version-controlled and reviewable
4. Quality gates (JaCoCo + Allure) are your safety net — enforce them, don't ignore them
5. Environment parity (Docker/Testcontainers) eliminates 'works on my machine' permanently
6. Diagnose bottlenecks before optimising — step timings in CI are your primary signal

Your Next Steps

This week

Apply the pipeline design from your capstone to a real project. Start with the unit-test job and add stages incrementally.

Next month

Enable JaCoCo coverage gate in production pipeline. Set initial threshold at current coverage – 5%, then tighten quarterly.

Next quarter

Introduce Testcontainers for your heaviest integration tests. Eliminate the shared staging dependency for at least one test suite.

Ongoing

Track pipeline duration and flake rate weekly. Review Allure trend charts in sprint retrospectives. Treat test health as a first-class engineering metric.

Resources

testcontainers.com · allurereport.org · rest-assured.io · docs.github.com/actions · jacoco.org · DORA (dora.dev)

Congratulations!

You have completed Continuous Testing with CI/CD — Intermediate

D1: CT Fundamentals	D2: Test Frameworks	D3: Version Control	D4: CI Pipeline Design
D5: API Testing	D6: Containers & Data	D7: Reports & Gates	**D8: Strategy & Capstone**

1ClickTech Global Services · Jai Singh